

Smart Digital Documentation System

SDDS — Complete Architecture, Feature Breakdown & Team Implementation Handbook

This comprehensive document is designed for all members of **Team Mavericks**. It explains the entire architecture, technologies used, security measures, explainable AI formulas, role-based workflows, and concrete step-by-step blueprints for future scalability and technical improvements.

Use this handbook to understand every part of the codebase, prepare for judge demonstrations, and answer technical questions with confidence.

Project Name: Smart Digital Documentation System (SDDS)

Repository: Harshal-112/HackathonProject

Live Demo: <https://smartdocumentatiion.netlify.app>

Team: Team Mavericks

Version: 1.0.0 Production Prototype

Target Users: Collector Offices, Tehsils, Panchayats, RTOs, Citizens

Table of Contents

1. Executive Summary & Problem Context	Section 1
2. Core Concepts Explained Simply (For Beginners)	Section 2
3. System Architecture & Complete Technology Stack	Section 3
4. In-Depth Features & End-to-End Workflow	Section 4
5. Explainable AI (XAI) & Privacy Engineering	Section 5
6. Security, RBAC & Database Transaction RPCs	Section 6
7. Database Schema & Migration Architecture	Section 7
8. Future Improvements — Step-by-Step Implementation Blueprints	Section 8
9. Team Demo Guide & Winning Hackathon Pitch Script	Section 9

1. Executive Summary & Problem Context

1.1 The Real-World Problem

In Indian government administrative offices (District Collectorates, Tehsil Offices, Zilla Parishads, Municipal Corporations, and Gram Panchayats), citizens submit thousands of physical documents every month — ranging from 7/12 Land Extracts, Caste Certificates, Income Proofs, Birth/Death Certificates, to Vehicle Registrations.

Currently, this workflow suffers from critical bottlenecks:

- **Manual Data Entry & Errors:** Clerks spend hours manually reading scans, typing names, Aadhaar numbers, survey numbers, and dates.
- **Slow Department Routing:** Applications sit on the wrong officer's desk for weeks before being redirected to the right department.
- **Document Fraud & Fake Papers:** Tampered certificates with fake stamps are difficult for verifiers to distinguish without public verification registries.
- **Data Privacy & PII Leakage:** Sensitive citizen data (Aadhaar numbers, PAN cards, phone numbers) are frequently uploaded to third-party public AI clouds without redaction.
- **Lack of Transparency & Auditability:** When an application is wrongly approved or rejected, there is no permanent tamper-evident log of who made the decision and why.

1.2 The SDDS Solution

SDDS (Smart Digital Documentation System) is an enterprise-grade digital document processing and verification platform designed specifically for Maharashtra's administrative workflow. It combines **in-browser WebAssembly multilingual OCR**, **automatic department routing**, **Explainable AI (XAI) with transparent mathematical scoring**, **automatic PII masking**, **strict Department-Scoped Role-Based Access Control (RBAC)**, and **tamper-proof QR-based public verification**.

Key Architecture Achievement: SDDS functions in two distinct modes:

- **Normal Mode:** Enhanced cloud AI via a secure server-side Supabase Edge Function that sanitizes PII before sending text to Gemini.
- **Confidential Mode:** 100% offline, zero-network-leak local processing using WebAssembly (WASM) running inside the user's browser.

2. Core Concepts Explained Simply

OCR Optical Character Recognition

Technology that takes an image (scanned certificate, photo of a document) and reads the text inside it, converting visual pixels into readable, editable digital text.

WASM WebAssembly

A high-speed binary code format that lets complex programs (like the C++ Tesseract OCR engine) run directly inside Google Chrome or Firefox without needing any backend server.

XAI Explainable AI

Unlike "black box" AI that gives an answer without reasons, XAI shows exact mathematical weights (e.g. 35% Keyword Match + 20% Structure) explaining *why* a document was classified.

PII Personally Identifiable Info

Sensitive private data belonging to a citizen (12-digit Aadhaar, 10-character PAN, Phone, Email) that must be automatically masked to prevent identity theft.

RBAC Role-Based Access Control

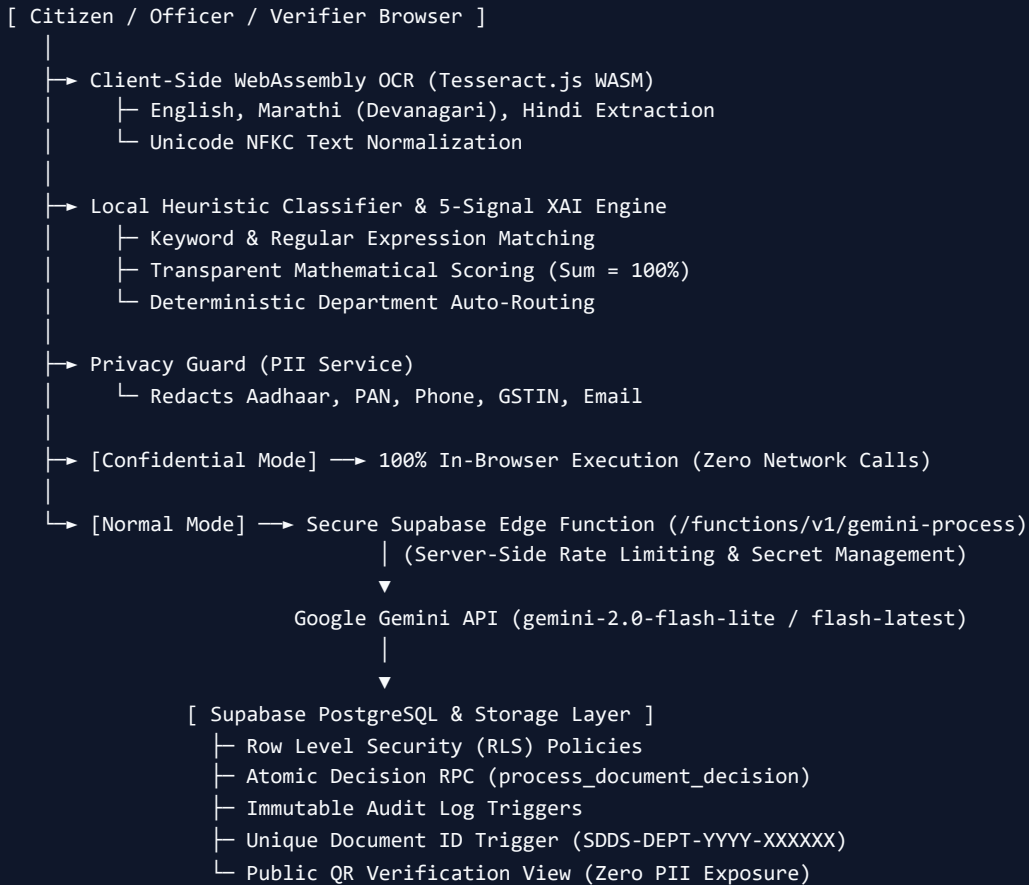
A security rule where users only see what their role allows. E.g., Revenue Verifiers can only approve Revenue documents; Admins can monitor but cannot verify.

RPC Remote Procedure Call

A secure database-level function written in PostgreSQL that runs directly on the database server inside an atomic transaction with row locking.

3. System Architecture & Complete Technology Stack

3.1 High-Level Architecture Diagram



3.2 Complete Technology Breakdown

Layer / Component	Technology Used	Exact Purpose in Project
Frontend Framework	React 18 + Vite 5	Blazing fast single-page application with hot module replacement and optimized production bundle.
Styling & UI	Tailwind CSS + Lucide Icons	Modern responsive UI with government-tailored dark/light themes, badges, and intuitive navigation.
Animations	Framer Motion	Smooth page transitions, drawer animations, and floating assistant micro-interactions.
In-Browser OCR	Tesseract.js v7 (WASM)	Optical character recognition running directly in the browser with Devanagari (Marathi/Hindi) support.
Cloud AI Processing	Google Gemini API (via Edge Function)	Metadata enhancement, summary generation, and natural language document analysis.
Backend Gateway	Supabase Edge Functions (Deno/TS)	Server-side API key protection, rate limiting (30 RPM), and multi-model fallback retry logic.
Database & Auth	Supabase PostgreSQL 15	Relational database, user authentication, Realtime subscriptions, and Row-Level Security.
QR Verification	QRCode.react + SVG	Generates dynamic cryptographic QR codes linking to public verification endpoints.
Testing Framework	Vitest	Automated unit testing suite executing 27 tests covering PII, XAI, RBAC, and workflow benchmarks.

4. In-Depth Features & End-to-End Workflow

4.1 Feature 1: Multi-Lingual Ingestion & WebAssembly OCR

- Supports standard formats: **PDF, JPG, PNG, DOCX** up to 10 MB.
- Uses `tesseract-core-simd-lstm.wasm` for accelerated matrix operations.
- Handles Devanagari script for official Marathi/Hindi government orders, 7/12 land records, and Gram Panchayat certificates.

4.2 Feature 2: Privacy Modes (Normal vs Confidential Mode)

- **Normal Mode:** Document text is masked of PII and sent via the secure Supabase Edge Function to Gemini for metadata enrichment.
- **Confidential Mode:** All external API network calls are completely blocked. The document is classified, routed, and processed entirely inside the browser's memory.

4.3 Feature 3: Automated Department Routing

SDDS automatically analyzes document keywords and routes files to the designated government department:

Department	Key Matching Keywords / Triggers	Target Authority
Revenue Department	7/12 Extract, 8A Extract, Mutation Entry, Ferfar, Jamabandi, Land Survey	Tehsildar / Talathi
District Collectorate	Caste Certificate, Domicile, Non-Creamy Layer, Election Order, Arms License	Sub-Divisional Officer / Collector
Municipal Corporation	Birth Certificate, Death Certificate, Property Tax, Building Permission	Ward Officer / Municipal Commissioner
Gram Panchayat	Gram Sabha Resolution, Namuna 8, Village Assessment, Water Tax	Gram Sevak / Sarpanch
RTO / Transport	Driving License, RC Book, Vehicle Fitness, Pollution Under Control (PUC)	RTO Inspector
Agriculture & Health	Kisan Credit Card (KCC), Crop Insurance, Disability Certificate, PMJAY	Agri Officer / Civil Surgeon

4.4 Feature 4: Role-Based Access Control (RBAC) Matrix

SDDS enforces strict segregation of duties to prevent administrative fraud:

User Role	Can Upload?	Can Approve / Reject?	Can Flag Re-verification?	Can Manage Users?
Admin	✔ Yes	✗ Strictly No (Monitor-Only)	✔ Yes (With reason & reassign)	✔ Yes (Activate, Assign Dept, Suspend)
Verifier	✔ Yes	✔ Yes (Only assigned in own dept)	✗ No	✗ No
Officer	✔ Yes	✗ No (Analytics & View Only)	✗ No	✗ No
Citizen	✔ Yes	✗ No (Only own uploads)	✗ No	✗ No

5. Explainable AI (XAI) & Privacy Engineering

5.1 The 5-Signal XAI Formula (Deterministic 100% Sum)

Unlike commercial black-box models, the SDDS XAI engine calculates an exact, transparent confidence score based on 5 mathematical feature signals whose weights strictly sum to **1.00 (100%)**:

$$\text{Total Confidence} = \sum_{i=1}^5 (S_i \times W_i)$$

Signal ID	Feature Name	Weight (W _i)	Evaluation Logic (S _i)
keywordScore	Keyword Match Frequency	0.35 (35%)	Ratio of matched domain-specific keywords found in OCR text.
patternScore	Official Format & ID Regex	0.20 (20%)	Presence of standard ID structures (e.g. MH-XX-YYYY-XXXXXX).
structureScore	Document Layout & Density	0.15 (15%)	Presence of standard headers ("Government of Maharashtra", Emblems).
ocrScore	OCR Engine Confidence	0.15 (15%)	Tesseract glyph recognition confidence percentage.
deptMatchScore	Department Category Alignment	0.15 (15%)	Match between predicted category and issuing government department.
Total Sum		1.00 (100%)	Deterministic, Auditable Output

5.2 PII Auto-Masking Patterns

The piiService.js module performs Unicode NFC normalization and applies OCR-tolerant regular expressions to sanitize sensitive data before any AI processing or display:

- Aadhaar Card:** Tolerant regex `\b[2-9]\d{3}[\s\-_]?[0-9011]{4}[\s\-_]?[0-9011]{4}\b` $\rightarrow$ `XXXX-XXXX-1234`
- PAN Card:** Regex `\b[A-Z]{5}[0-9]{4}[A-Z]\b` $\rightarrow$ `XXXXX1234X`
- GSTIN:** Regex `\b\d{2}[A-Z]{5}\d{4}[A-Z]{1}[A-Z\d]{1}[Z]{1}[A-Z\d]{1}\b` $\rightarrow$ `27XXXXX1234XZX`
- Mobile & Email:** Replaced with standard compliance redaction masks.

6. Security, RBAC & Database Transaction RPCs

6.1 Atomic Decision RPC: process_document_decision

To prevent race conditions where two verifiers might act on the same document simultaneously, decision making is executed via a PostgreSQL SECURITY DEFINER function using row-level locking (FOR UPDATE):

```

-- Highlights from supabase/migrations/003_document_decision_rpc.sql
CREATE OR REPLACE FUNCTION public.process_document_decision(
  p_document_id uuid,
  p_decision text,    -- 'approve', 'reject', 'request_changes'
  p_remarks text DEFAULT ''
)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_doc record;
  v_caller record;
BEGIN
  -- 1. Row lock to eliminate race conditions
  SELECT * INTO v_doc FROM public.documents WHERE id = p_document_id FOR UPDATE;

  -- 2. Verify caller role and department match
  SELECT * INTO v_caller FROM public.profiles WHERE id = auth.uid();
  IF v_caller.role != 'verifier' OR v_caller.department != v_doc.department THEN
    RAISE EXCEPTION 'Unauthorized: Only the assigned departmental verifier can act on this document.';
  END IF;

  -- 3. Execute atomic state update & log immutable audit record
  UPDATE public.documents
  SET status = CASE WHEN p_decision = 'approve' THEN 'approved' ELSE 'rejected' END,
      updated_at = now()
  WHERE id = p_document_id;

  RETURN jsonb_build_object('success', true, 'status', v_doc.status);
END;
$$;

```

7. Database Schema & Migration Architecture

TABLE profiles

- `id` (UUID, references `auth.users`)
- `name` (text), `email` (text)
- `role` (admin, verifier, officer, citizen)
- `department` (revenue, collector, etc.)
- `status` (active, pending, suspended)

TABLE documents

- `id` (UUID), `document_number` (SDDS-DEP-YYYY-XXXXXX)
- `title`, `file_name`, `file_url`, `file_size`
- `category`, `department`, `priority`
- `status` (pending, approved, rejected, re_verification)
- `assigned_verifier_id`, `ocr_text`, `metadata` (JSONB)

TABLE audit_logs

- `id` (UUID), `user_id`, `user_name`, `role`
- `action` (UPLOAD, APPROVE, REJECT, FLAG_REVERIFY)
- `entity_type`, `entity_id`, `details` (JSONB)
- `ip_address`, `created_at` (Immutable: NO UPDATE/DELETE)

VIEW public_document_verifications

- Exposes ONLY: `id`, `title`, `status`, `document_number`, `department`, `category`, `created_at`, `is_authentic`
- **Completely strips:** uploader identity, raw OCR text, internal notes, and storage URLs.

8. Future Improvements — Implementation Blueprints

The project README lists 5 key future improvements. Here is the exact technical blueprint for how your team can implement each one:

8.1 Improvement 1: Document-Specific ML / Layout Models (Donut & LayoutLMv3)

Goal: Replace basic OCR keyword extraction with deep-learning visual-document understanding for complex tables and distorted forms.

- **Architecture:** Deploy a Python FastAPI microservice running `naver-clova-ix/donut-base` or `microsoft/layoutlmv3-base` packaged as an ONNX runtime container.
- **How to implement:**
 1. Train a fine-tuned Donut model on 500 annotated 7/12 Land Extracts and Caste Certificates.
 2. Expose endpoint `POST /api/v1/extract-document` accepting multipart image data.
 3. Update `aiService.js` in frontend to call this endpoint when high-precision table extraction is required.

8.2 Improvement 2: Digital Signatures & Blockchain Tamper-Proof Anchors

Goal: Guarantee mathematical proof that an approved certificate has not been altered after issuance.

- **How to implement:**
 1. When a verifier approves a document in `process_document_decision`, compute a **SHA-256 cryptographic digest** of the document metadata + certificate number: `digest = SHA256(doc_number + citizen_id + issue_date + status)`.
 2. Sign the digest using the Verifier's private PKI key (e-Sign standard).
 3. Anchor the hash to a public/consortium blockchain (e.g. Polygon PoS or Hyperledger Fabric) by emitting an on-chain event: `DocumentVerified(bytes32 docHash, string docNumber, uint256 timestamp)`.

4. On the public QR verification page (`/verify/:id`), compare the live document hash with the immutable blockchain ledger anchor.

8.3 Improvement 3: Asynchronous High-Volume Worker Queue (BullMQ / Redis)

Goal: Handle batch uploads of 500+ documents at once during large-scale government digitization drives without browser timeouts.

- **How to implement:**

1. Deploy a Redis-backed **BullMQ** worker service in Node.js.
2. Frontend uploads files directly to Supabase Storage and pushes job tickets to `document-processing-queue`.
3. Background worker processes OCR, PII masking, and classification asynchronously and writes results to PostgreSQL.
4. Frontend receives real-time progress updates via Supabase Realtime WebSocket events on the `documents` table.

8.4 Improvement 4: Native Multilingual UI Localization (i18next)

Goal: Seamless 1-click UI switching between English (English), Marathi (मराठी), and Hindi (हिंदी).

- **How to implement:**

1. Install `i18next` and `react-i18next` in `project/package.json`.
2. Create locale JSON files: `src/locales/mr.json`, `src/locales/hi.json`, `src/locales/en.json`.
3. Wrap labels with `const { t } = useTranslation()` (e.g. `t('dashboard.title')`).
4. Add a language selector dropdown in `navbar.jsx` alongside the theme toggle.

9. Team Demo Guide & Winning Hackathon Pitch Script

9.1 Demo Personas & Credentials

Use these test personas during your presentation to demonstrate the RBAC separation:

Persona	Role	Assigned Department	Key Action to Show Judges
Admin	admin	All Departments	Show Monitor-Only queue, User Management, and "Flag for Re-verification".
Suresh (Verifier)	verifier	Revenue Department	Approve/Reject 7/12 Land Record assigned specifically to him.
Ramesh (Citizen)	citizen	Public Citizen	Upload document in Confidential Mode, verify QR code without seeing internal notes.

9.2 The 3-Minute Winning Presentation Script

Step 1: The Hook (0:00 - 0:30)

"Respected judges, every year over 50 million citizen certificates are processed in Maharashtra. Clerks are overwhelmed by paper, fraud is rampant, and citizen privacy is compromised. We present **SDDS** — an AI-assisted, privacy-first digital documentation and verification portal built specifically for Indian administrative workflows."

Step 2: Live Ingestion & Privacy Mode (0:30 - 1:15)

"Watch this live: We upload a Marathi 7/12 Land Extract. Notice our **Confidential Mode** toggle. When activated, WebAssembly runs OCR and classification 100% locally in the browser — not a single byte leaves the machine. Aadhaar and PAN numbers are automatically redacted using our PII engine."

Step 3: Explainable AI & RBAC (1:15 - 2:00)

"Unlike black-box AI, our **XAI Engine** shows verifiers the exact mathematical breakdown (35% Keywords + 20% ID Regex + 15% Structure = 100%). In our RBAC, Admins cannot verify documents — this prevents corruption. Verifiers use database-locked atomic transactions (`process_document_decision`) to approve records."

Step 4: Public Verification & Conclusion (2:00 - 3:00)

"Once approved, a secure QR code is generated. Any citizen or bank officer can scan it to instantly verify authenticity. By enforcing **Data Minimization**, the public page only shows certificate validity — zero citizen PII or officer remarks are leaked. SDDS makes e-Governance faster, safer, and fully accountable."